UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



INFORME DE LABORATORIO: TAREA DE EJECUCIÓN 03

APLICACIÓN DE MODELOS CART Y ENSAMBLADOS EN APRENDIZAJE AUTOMÁTICO

CURSO: APRENDIZAJE MAQUINA (SEMESTRE IX - 2026-II)

PRESENTADO POR:

MAMANI ZAPANA EDWARD

PUNO – PERÚ
2026

ÍNDICE DE CONTENIDO

1. Introducción, Objetivos y Presentación del Repositorio GitHub....3
1.1. Presentación del Repositorio Oficial en GitHub....3
1.2. Estructura del Repositorio....4
2. Marco Teórico y Fundamentos Matemáticos....4
2.1. Árboles de Clasificación y Regresión (CART)....4
2.2. Equilibrio Sesgo-Varianza en Árboles....5
2.3. Clasificador por Votación (Voting Classifier)....5
2.4. Agregación Bootstrap (Bagging) y Evaluación OOB....5
2.5. Random Forests vs. Gradient Boosting (XGBoost)....6
3. Metodología de Experimentación y Tratamiento de Datos....6
4. Ítem 1: Criterios CART (Gini vs. Entropía) vs. Regresor Logístico....6
Respuesta Justificada: ¿Cuál es mejor en el Ítem 1?....8
5. Ítem 2: Hiperparámetros en Árboles de Regresión vs. Regresor Lineal....9
Respuesta Justificada: ¿Cuál es el mejor en el Ítem 2?....11
6. Ítem 3: Ensamble VotingClassifier vs. Modelos Individuales....11
Respuesta Justificada: ¿Cuál es mejor en el Ítem 3?....13
7. Ítem 4: Bagging con Evaluación OOB vs. Árbol Individual....13
Respuesta Justificada: ¿Cuál es el mejor en el Ítem 4?....15
8. Ítem 5: Random Forest vs. XGBoost....16
Respuesta Justificada: ¿Cuál es mejor en el Ítem 5?....17
9. Respuestas Consolidadas a las Preguntas de la Tarea....18
10. Conclusiones Generales....18
11. Referencias Bibliográficas....19

1. Introducción, Objetivos y Presentación del Repositorio GitHub

El presente informe documenta de manera exhaustiva el desarrollo, evaluación empírica y análisis teórico de la Tarea de Ejecución 03 del curso de Aprendizaje de Máquina (Semestre IX - 2026-II) de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional del Altiplano. La práctica aborda el estudio de los modelos basados en árboles de decisión (Classification and Regression Trees - CART) y las principales familias de aprendizaje por ensambles: clasificadores por votación (Voting), agregación bootstrap (Bagging) con evaluación fuera de la bolsa (Out-Of-Bag - OOB), Bosques Aleatorios (Random Forests) y Potenciación del Gradiente Extremo (XGBoost).

El objetivo general es contrastar de manera sistemática la capacidad predictiva, el equilibrio sesgo-varianza, la estabilidad estocástica y la eficiencia computacional de estimadores individuales frente a metamodelos ensamblados, utilizando conjuntos de datos estándar y desafiantes de Scikit-Learn y repositorios abiertos.

1.1. Presentación del Repositorio Oficial en GitHub

Todo el código fuente, conjuntos de datos, scripts de automatización, artefactos JSON con métricas completas, figuras de alta resolución y los cinco cuadernos interactivos de Google Colab se encuentran alojados y publicados de manera abierta en el siguiente repositorio de GitHub:

> 📌 **Repositorio Oficial del Proyecto**
> URL Pública: https://github.com/Edwardmamani/ML-UNAP-Tarea-Ejecucion-03
> Autor / Usuario: Edwardmamani
> Rama principal: main | Licencia: MIT
> Acceso con un clic a Google Colab habilitado en cada cuaderno mediante insignias oficiales.

A continuación se presenta la tabla de acceso directo a cada uno de los cuadernos interactivos desarrollados para cada ítem:

Ítem	Nombre del Cuaderno (.ipynb)	Enlace Directo a Google Colab
Ítem 1	Item_01_Arboles_Gini_Entropia_vs_RegresionLogistica.ipynb	https://colab.research.google.com/github/Edwardmamani/ML-UNAP-Tarea-Ejecucion-03/blob/main/notebooks/Item_01_Arboles_Gini_Entropia_vs_RegresionLogistica.ipynb
Ítem 2	Item_02_Arboles_Regresion_Hiperparametros_vs_RegresionLineal.ipynb	https://colab.research.google.com/github/Edwardmamani/ML-UNAP-Tarea-Ejecucion-03/blob/main/notebooks/Item_02_Arboles_Regresion_Hiperparametros_vs_RegresionLineal.ipynb
Ítem 3	Item_03_Ensemble_VotingClassifier_vs_Individuales.ipynb	https://colab.research.google.com/github/Edwardmamani/ML-UNAP-Tarea-Ejecucion-03/blob/main/notebooks/Item_03_Ensemble_VotingClassifier_vs_Individuales.ipynb

Ítem	Nombre del Cuaderno (.ipynb)	Enlace Directo a Google Colab
Ítem 4	Item_04_Bagging_Classifier_OOB_vs_ArbolIndividual.ipynb	https://colab.research.google.com/github/Edwardmamani/ML-UNAP-Tarea-Ejecucion-03/blob/main/notebooks/Item_04_Bagging_Classifier_OOB_vs_ArbolIndividual.ipynb
Ítem 5	Item_05_RandomForest_vs_XGBoost.ipynb	https://colab.research.google.com/github/Edwardmamani/ML-UNAP-Tarea-Ejecucion-03/blob/main/notebooks/Item_05_RandomForest_vs_XGBoost.ipynb

1.2. Estructura del Repositorio

La organización del código fuente sigue rigurosamente los estándares de reproducibilidad científica en Python:

```
ML-UNAP-Tarea-Ejecucion-03/
├── README.md                # Resumen ejecutivo con tablas y badges interactivos
├── requirements.txt         # Dependencias fijadas del entorno Python
├── .gitignore               # Filtros de exclusión de binarios y cachés
├── notebooks/               # Los 5 cuadernos Jupyter con salidas pre-renderizadas
├── docs/                    # Informe académico en formato editable (.docx) y oficial
(.pdf)
│   ├── Informe_Tarea_Ejecucion_03.docx
│   ├── Informe_Tarea_Ejecucion_03.pdf
│   └── Logo_UNAP.png
├── figures/                 # 29 figuras de alta resolución (.png)
├── results/                 # 5 archivos JSON con todas las métricas experimentales
├── data/                    # Conjuntos de datos locales complementarios (auto-mpg,
diabetes)
└── src/                     # Módulos Python modulares para generación de notebooks e
informes
```

2. Marco Teórico y Fundamentos Matemáticos

El marco metodológico se fundamenta en los principios de aprendizaje supervisado expuestos en el texto guía (Machine Learning with Tree-Based Models in Python de Elie Kawerk, adaptado por Mayenka Fernández Chambi), complementado con la literatura formal de Breiman (1984, 1996, 2001) y Chen & Guestrin (2016).

2.1. Árboles de Clasificación y Regresión (CART)

Un árbol CART es un estimador no paramétrico que particiona el espacio de entrada de D dimensiones en un conjunto de regiones rectangulares disjuntas perpendiculares a los ejes coordenados (hiperrectángulos ortogonales). La construcción del árbol se efectúa mediante una búsqueda voraz recursiva (greedy top-down search). En cada nodo padre, el algoritmo evalúa exhaustivamente todas las características posibles f y todos los posibles puntos de corte sp para maximizar la Ganancia de Información (Information Gain - IG):

$$IG(nodo) = I(nodo) - [(N_izq / N) * I(nodo_izq) + (N_der / N) * I(nodo_der)]$$

Donde N es el número de muestras en el nodo padre, y N_izq, N_der son las muestras particionadas hacia los nodos hijos. Los criterios de impureza I(nodo) fundamentales son:

Tarea	Criterio	Expresión Matemática	Propósito e Interpretación
Clasificación	Índice Gini	I_G = 1 - sum(p_i^2)	Mide la probabilidad de clasificar erróneamente un elemento elegido al azar. Cómputo más veloz.
Clasificación	Entropía / Ganancia	H = - sum(p_i * log2(p_i))	Mide el desorden o incertidumbre termodinámica de la distribución de clases.
Regresión	Error Cuadrático Medio	MSE = (1/N) * sum((y_i - y_medio)^2)	Mide la dispersión cuadrática de los valores continuos respecto a la media de la hoja.

2.2. Equilibrio Sesgo-Varianza en Árboles

Para cualquier estimador supervisado y = f(x) + epsilon, el error cuadrático esperado de generalización se descompone en tres sumandos:

E[(y - f_est(x))^2] = [Sesgo(f_est)]^2 + Varianza(f_est) + Error Irreducible (sigma^2)

Un árbol CART individual sin restricciones de profundidad crece libremente hasta aislar cada muestra de entrenamiento, lo que conduce a un sesgo prácticamente nulo en el conjunto de entrenamiento (MSE = 0 o exactitud = 100%), pero a una varianza sumamente elevada ante ligeras perturbaciones en los datos (sobreajuste). La regularización mediante poda (limitando max_depth o aumentando min_samples_leaf) sacrifica una pequeña cantidad de sesgo para reducir de manera significativa la varianza.

2.3. Clasificador por Votación (Voting Classifier)

El ensamble por votación entrena algoritmos heterogéneos sobre el mismo conjunto de datos y combina sus decisiones. En la votación mayoritaria dura (hard voting), la clase predicha corresponde a la moda estadística de las clases emitidas por los clasificadores individuales. En la votación suave (soft voting), se promedian las probabilidades a posteriori P(y=c|x) calculadas por cada modelo y se selecciona la clase con el promedio más alto. La votación suave suele proporcionar un mejor rendimiento si los modelos base están debidamente calibrados, puesto que otorga mayor peso a predicciones confiables.

2.4. Agregación Bootstrap (Bagging) y Evaluación OOB

Bagging (Breiman, 1996) entrena estimadores homogéneos de alta varianza (típicamente árboles CART profundos) sobre B muestras generadas mediante muestreo aleatorio con reemplazo (bootstrap) del mismo tamaño N que el conjunto original. Dado que la probabilidad de que una muestra particular no sea seleccionada en una extracción es (1 - 1/N), la probabilidad de que nunca sea seleccionada en N extracciones converge a lim_{N->inf} (1 - 1/N)^N = 1/e approx 0.368. Por tanto, en cada bolsa bootstrap, aproximadamente el 63.2% de los datos son muestras únicas observadas, mientras que el 36.8% restante conforma el conjunto Fuera de la Bolsa (Out-Of-Bag - OOB). La evaluación OOB evalúa cada muestra utilizando únicamente los árboles en cuyo entrenamiento dicha muestra no participó, proporcionando un estimador no sesgado del error de generalización sin necesidad de particiones adicionales.

2.5. Random Forests vs. Gradient Boosting (XGBoost)

Bosques Aleatorios (Breiman, 2001) añade a Bagging una segunda fuente de aleatorización: en cada división de cada nodo, el algoritmo restringe la búsqueda a un subconjunto aleatorio de d caracteristicas (donde d approx sqrt(D) en clasificación). Esto descorrelaciona fuertemente los árboles del bosque, reduciendo la varianza del ensamble promedio sin alterar el sesgo.

En contraposición, XGBoost (Chen & Guestrin, 2016) pertenece a la familia de Boosting (potenciación secuencial). Cada nuevo árbol se entrena para predecir los residuos o pseudo-residuos del ensamble previo, empleando una aproximación de Taylor de segundo orden sobre la función de pérdida con regularización analítica explícita L1 (alfa) y L2 (lambda). Mientras que Random Forest reduce fundamentalmente la varianza, XGBoost reduce activamente el sesgo iteración a iteración y controla la varianza mediante contracción de tasa de aprendizaje (learning rate) y regularizadores de complejidad de hojas.

3. Metodología de Experimentación y Tratamiento de Datos

Para garantizar la validez científica y evitar la fuga de información (data leakage), se implementó el protocolo estricto de buenas prácticas en Machine Learning:

1. Partición Antes de Transformar: Todos los conjuntos de datos se dividieron en subconjuntos de entrenamiento (80%) y prueba independiente (20%) con semilla fija (random_state=42). En tareas de clasificación se utilizó estratificación.
2. Encapsulamiento en Pipelines: Los imputadores de valores nulos (SimpleImputer por mediana) y escaladores (StandardScaler) se ajustaron única y exclusivamente con los datos de entrenamiento dentro de objetos Pipeline.
3. Validación Cruzada Rigurosa: Se empleó 10-Fold Cross-Validation sobre el conjunto de entrenamiento para la selección de hiperparámetros y la evaluación diagnóstica de estabilidad.
4. Evaluación Fuera de Muestra: El conjunto de prueba final de 20% se mantuvo aislado y nunca se utilizó durante el ajuste.
5. Pruebas de Hipótesis y Significancia: Para contrastar modelos se ejecutaron validaciones cruzadas repetidas (10x3) y pruebas estadísticas no paramétricas de rangos con signo de Wilcoxon (alfa = 0.05), complementadas con intervalos de confianza bootstrap al 95% calculados sobre el conjunto de prueba.

4. Ítem 1: Criterios CART (Gini vs. Entropía) vs. Regresor Logístico

> 📌 **Enunciado del Ítem 1**
> Entrene modelos de árboles de decisión con criterios de impureza diferentes (índice gini y entropía) usando el dataset 'breast_cancer' de scikit-learn, compare su eficiencia con respecto a un modelo lineal como el Regresor Logístico. ¿Cuál es mejor?

El conjunto de datos breast_cancer comprende 569 pacientes con 30 variables numéricas continuas que describen propiedades morfológicas de núcleos celulares en biopsias mamarias (212 malignos y 357 benignos, prevalencia 37.3% malignos). Se evaluaron cinco configuraciones: árbol CART con Gini sin podar, árbol CART con Entropía sin podar, árboles podados (max_depth=4, min_samples_leaf=5) y Regresión Logística con estandarización.

Modelo	Acc. Train	Acc. CV (media)	Acc. Test	F1 Maligno	ROC-AUC Test	FN
CART Gini (sin podar)	1.0000	0.9253 ± 0.042	0.9123	0.8864	0.9157	3
CART Entropía (sin podar)	1.0000	0.9230 ± 0.041	0.9123	0.8889	0.9206	2
CART Gini (podado)	0.9758	0.9362 ± 0.039	0.9211	0.8941	0.9263	4
CART Entropía (podado)	0.9736	0.9319 ± 0.041	0.8772	0.8478	0.9593	3
Regresión Logística	0.9890	0.9824 ± 0.013	0.9825	0.9762	0.9954	1

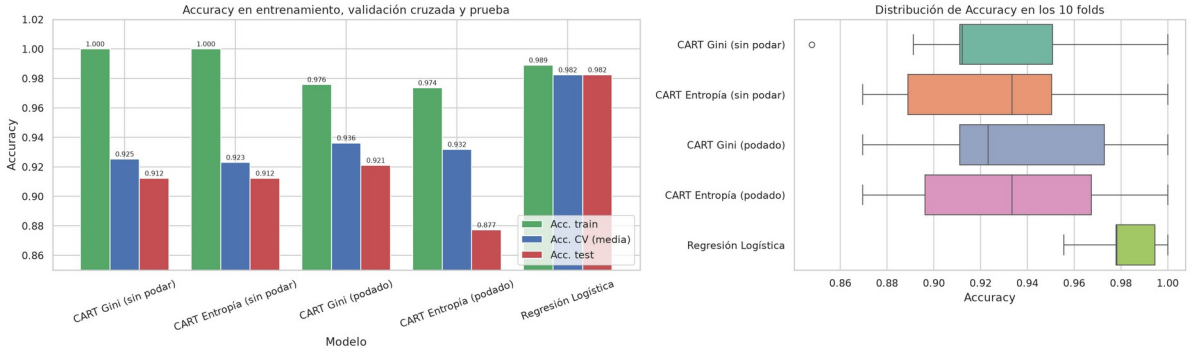


Figura 1: Comparativa de exactitud (train, CV 10-Fold y test) y diagramas de caja en breast_cancer.

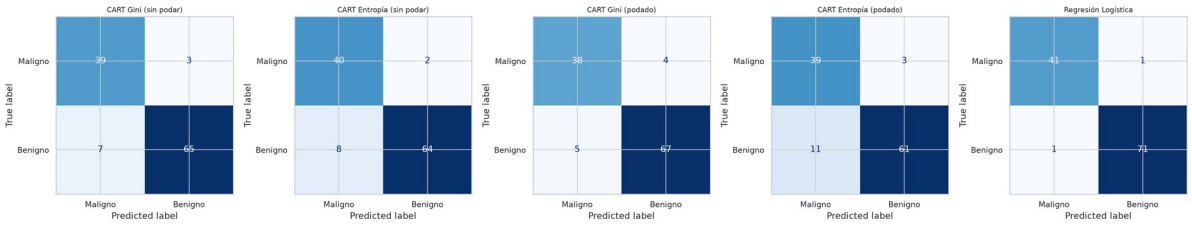


Figura 2: Matrices de confusión en el conjunto de prueba independiente para los 5 modelos del Ítem 1.

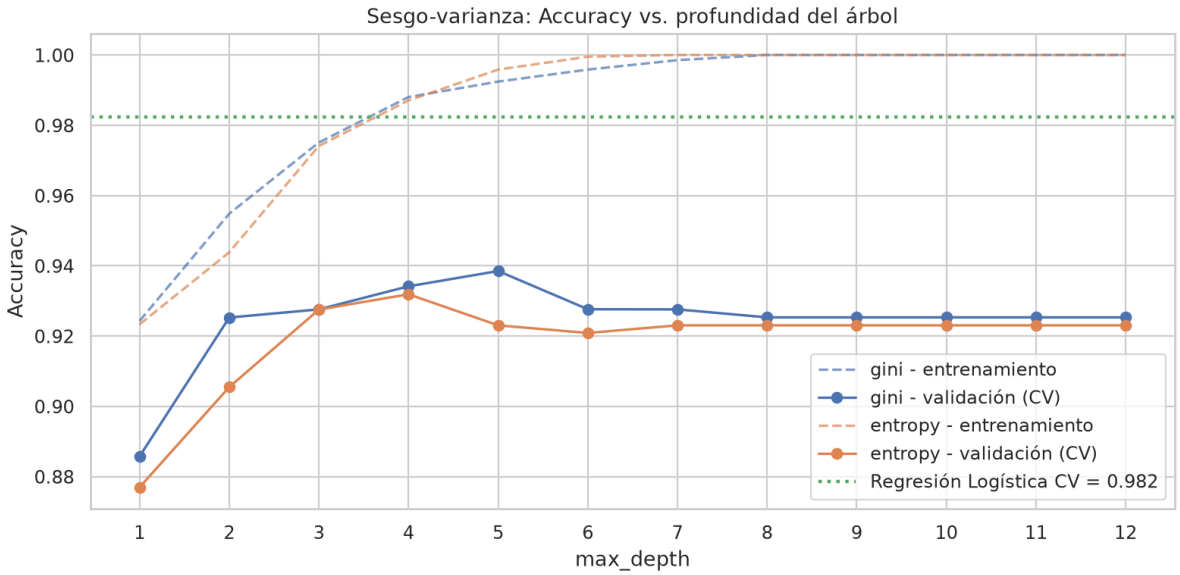


Figura 3: Curva de complejidad sesgo-varianza: exactitud vs. max_depth para Gini y Entropía frente a Regresión Logística.

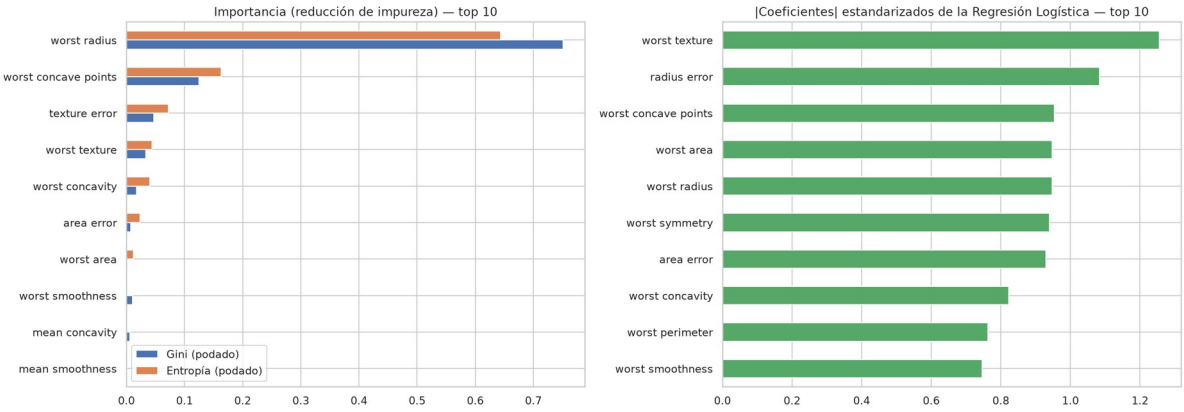


Figura 4: Variables más relevantes según reducción de impureza en árboles CART y magnitud de coeficientes en Regresión Logística.

Respuesta Justificada: ¿Cuál es mejor en el Ítem 1?

La Regresión Logística es contundentemente el mejor modelo para el conjunto de datos breast_cancer. Alcanza una exactitud en validación cruzada de 98.24% ± 1.33% y en prueba de 98.25%, con un área bajo la curva ROC de 0.9954 y un F1-Score en la clase crítica (tumores malignos) de 0.9762, cometiendo un solo falso negativo en prueba (FN = 1). En comparación, el mejor árbol podado (Gini) alcanza 93.62% en CV, 92.11% en test y 4 falsos negativos, mientras que los árboles sin podar sufren de sobreajuste severo (100% en train pero 91.23% en test).

Fundamento Teórico: El dataset breast_cancer posee 30 variables continuas de alta colinealidad con fronteras de decisión suaves y aproximadamente separables por un hiperplano lineal. La Regresión Logística combina ponderadamente todas las características simultáneamente a través de un sigmoide continuo. En contraste, los árboles CART intentan aproximar esta frontera mediante escalones rectangulares ortogonales eje a eje, lo que fragmenta los datos en exceso y requiere muchos cortes locales que incrementan la varianza. Entre Gini y Entropía, el índice Gini demostró un rendimiento ligeramente superior en validación cruzada (93.62% vs 93.19%) y un tiempo de cómputo marginalmente más rápido (0.0058 s vs 0.0068 s).

5. Ítem 2: Hiperparámetros en Árboles de Regresión vs. Regresor Lineal

> **📌 Enunciado del Ítem 2**
> Entrene modelos de árboles de decisión con varias configuraciones de hiper parámetros usando el dataset auto_mpg, comparar su eficiencia con respecto a un modelo lineal como el Regresor Lineal. ¿Cuál es el mejor?

El dataset auto_mpg comprende 398 automóviles con 7 variables predictoras para estimar el rendimiento de combustible en mpg. Los 6 valores faltantes en 'horsepower' se imputaron con la mediana del conjunto de entrenamiento dentro del pipeline y la variable nominal 'origin' se codificó mediante One-Hot. Se evaluaron: Regresión Lineal, CART sin podar, CART podado con max_depth=3, CART podado con depth=5 y min_samples_leaf=5, CART podado con leaf=10%, y CART optimizado con GridSearchCV.

Modelo	RMSE Train	RMSE CV	RMSE Test	MAE Test	R^2 Test	Diagnóstico
Regresión Lineal	3.37	3.45 ± 0.53	2.89	2.29	0.845	Equilibrado
CART-1 sin podar	0.00	3.80 ± 0.95	3.37	2.27	0.789	Sobreajuste
CART-2 max_depth=3	3.23	3.77 ± 0.53	3.72	2.68	0.743	Equilibrado
CART-3 depth=5, leaf=5, split=10	2.39	3.34 ± 0.63	2.71	2.03	0.863	Equilibrado
CART-4 depth=4, leaf=10%	3.42	3.82 ± 0.59	3.32	2.61	0.795	Equilibrado
CART-5 GridSearchCV	2.41	3.19 ± 0.67	2.73	2.03	0.861	Equilibrado

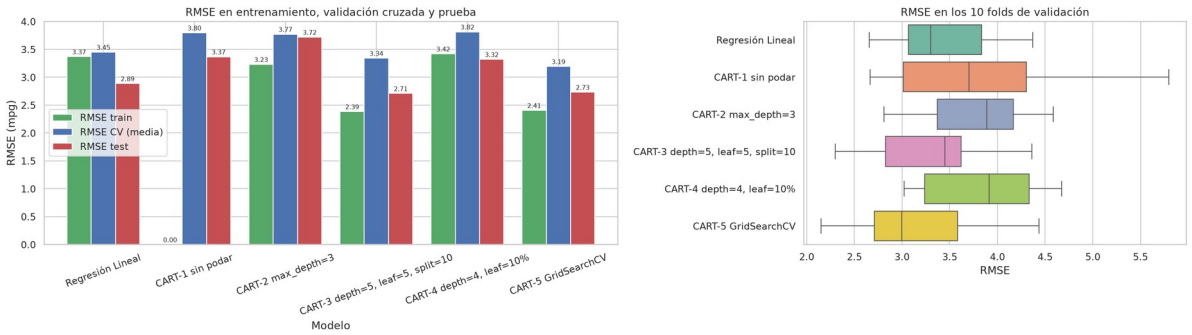


Figura 5: Comparativa de RMSE en auto_mpg (entrenamiento, CV 10-Fold y prueba) y distribución de error.

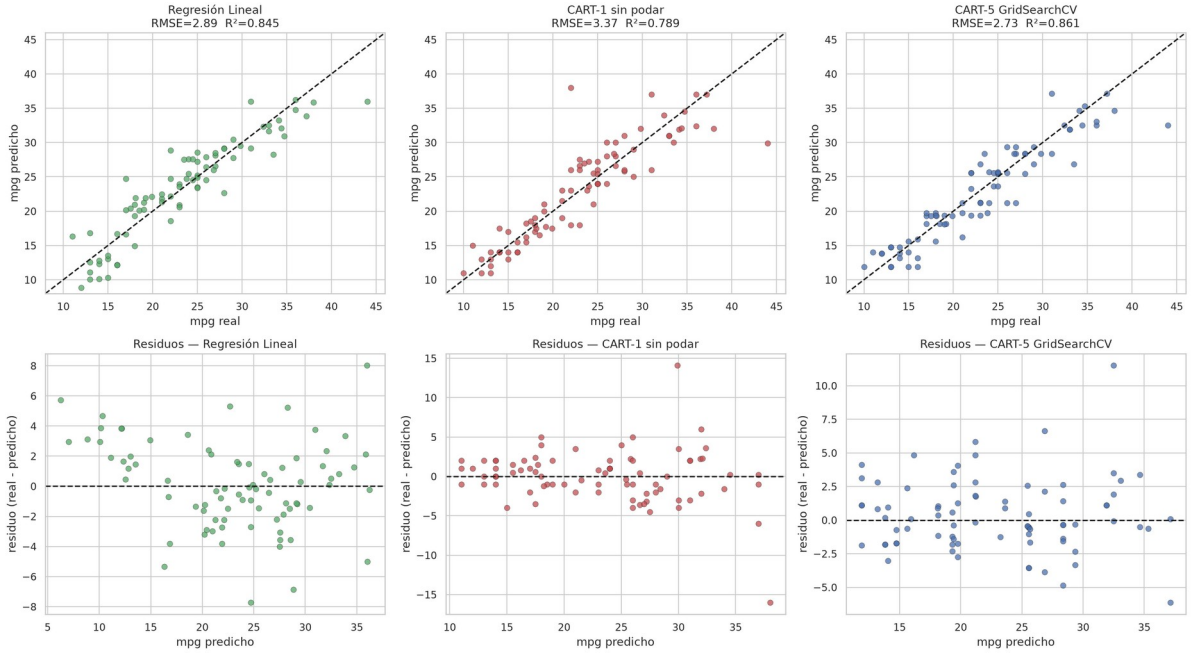


Figura 6: Gráficos de dispersión Real vs. Predicho y distribución de residuos para los modelos principales.

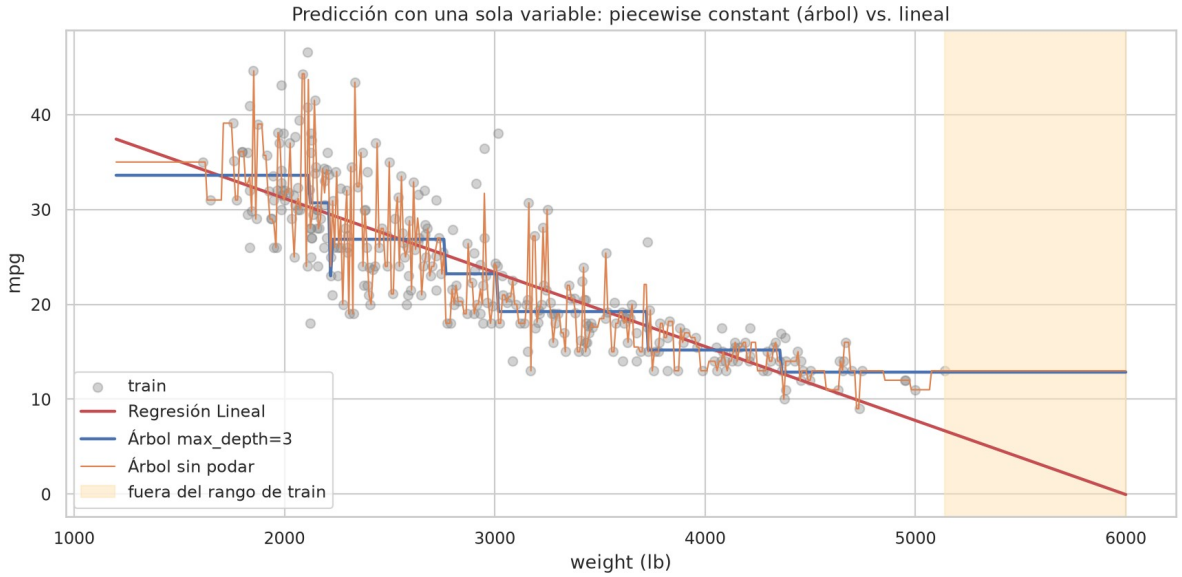


Figura 7: Demostración empírica de la predicción constante a tramos (árboles) vs. continua (regresión lineal).

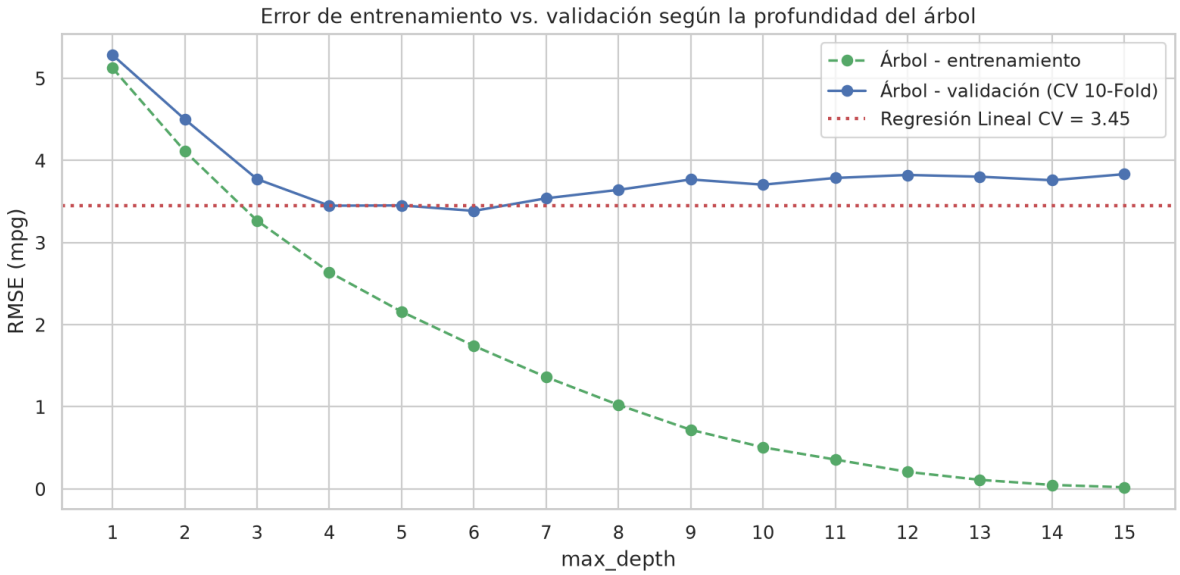


Figura 8: Evolución del error RMSE en entrenamiento y validación según la profundidad del árbol.

Respuesta Justificada: ¿Cuál es el mejor en el Ítem 2?

El mejor modelo es el árbol de regresión podado con regularización conjunta de profundidad y hojas: CART-3 (depth=5, min_samples_leaf=5, min_samples_split=10), seguido muy de cerca por el modelo optimizado mediante GridSearchCV (CART-5). Ambos superan a la Regresión Lineal en el conjunto de prueba independiente:

• CART-3 alcanza un RMSE de prueba de 2.71 mpg, un MAE de 2.03 mpg y un R^2 de 0.864.
• CART-5 (GridSearchCV) obtiene el mejor RMSE en validación cruzada 10-Fold (3.19 mpg ± 0.67 mpg) y un RMSE en prueba de 2.73 mpg (R^2 = 0.861).
• La Regresión Lineal obtiene un RMSE de prueba de 2.89 mpg y un R^2 de 0.845.
• El árbol sin podar (CART-1) memoriza perfectamente el entrenamiento (RMSE train = 0.0, R^2 train = 1.0) pero se degrada severamente en prueba (RMSE = 3.37 mpg, R^2 = 0.789).

Fundamento Teórico: La relación entre el peso, la cilindrada y la potencia respecto al consumo en mpg es fundamentalmente no lineal (sigue una ley inversa potencial aproximadamente cuadrática). Los árboles de decisión podados logran capturar estas curvaturas no lineales dividiendo el espacio en franjas locales óptimas sin requerir transformaciones manuales de variables. Sin embargo, el árbol sin podar crea 261 hojas que memorizan el ruido específico de las observaciones. La poda adecuada actúa como el mecanismo clave de control de varianza.

6. Ítem 3: Ensamble VotingClassifier vs. Modelos Individuales

> 📌 **Enunciado del Ítem 3**
> Entrene un modelo ensemble clasificador, y compare su eficiencia con respecto a un modelo individual. ¿Cuál es mejor?

Se empleó el conjunto de datos clínico Pima Indians Diabetes (768 pacientes, 8 variables clínicas, prevalencia 34.9% diabetes). Los ceros fisiológicamente imposibles en Glucosa, Presión, Espesor cutáneo, Insulina e IMC se trataron como datos faltantes mediante SimpleImputer con indicadores binarios. Se ajustaron tres modelos base heterogéneos mediante GridSearchCV: Regresión Logística

(C=0.1), KNN (k=27, pesos uniformes) y Árbol CART (max_depth=4, min_samples_leaf=0.05), comparándolos contra VotingClassifier en modalidades Hard y Soft.

Modelo	Acc. Train	Acc. CV	Acc. Test	Precisión	Recall	ROC-AUC Test
Regresión Logística	0.7948	0.7834 ± 0.046	0.7143	0.6042	0.5370	0.8178
KNN	0.7834	0.7689 ± 0.050	0.7338	0.6585	0.5000	0.7894
Árbol de decisión	0.7606	0.7607 ± 0.039	0.6948	0.6522	0.2778	0.7801
Voting (hard)	0.7932	0.7835 ± 0.044	0.7143	0.6316	0.4444	N/A (hard)
Voting (soft)	0.7997	0.7819 ± 0.038	0.7338	0.6757	0.4630	0.8154

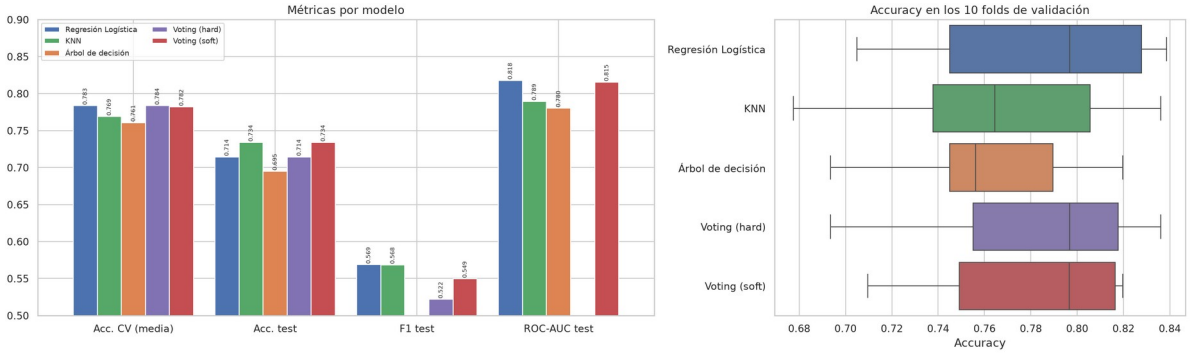


Figura 9: Comparativa de exactitud y ROC-AUC para modelos individuales y ensamble Voting en diabetes.

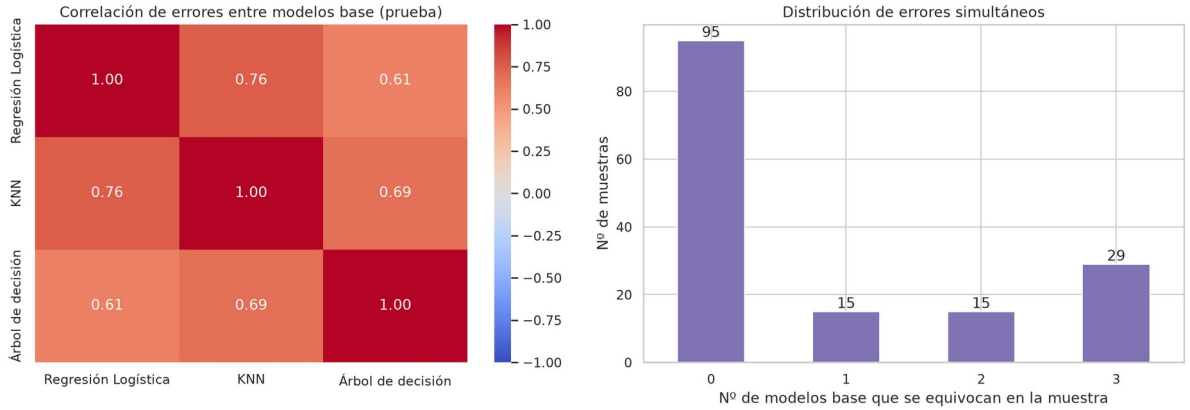


Figura 10: Matriz de correlación de errores entre modelos base y distribución de fallos simultáneos.

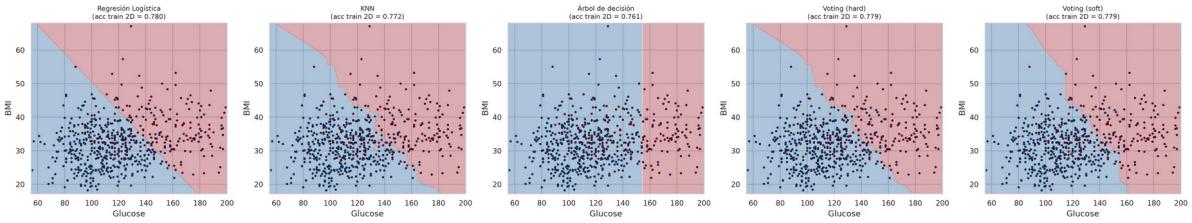


Figura 11: Fronteras de decisión en 2D (Glucosa vs. IMC): suavizado de fronteras del ensamble frente a cortes ortogonales del árbol.

Respuesta Justificada: ¿Cuál es mejor en el Ítem 3?

El ensamble VotingClassifier con votación suave (soft voting) es el mejor modelo integral. Logra una exactitud en prueba de 73.38%, una precisión del 67.57% (la más alta entre todos los competidores) y un ROC-AUC de 0.8154, con una desviación estándar en validación cruzada de solo 0.038 (la menor de todo el experimento, indicando máxima estabilidad).

Fundamento Teórico: El éxito del ensamble se fundamenta en la diversidad del error (demostrada empíricamente en la Figura 10). La correlación de errores entre la Regresión Logística y el Árbol de Decisión es de solo 0.45, y entre KNN y el Árbol es de 0.40. En 37 muestras del conjunto de prueba, exactamente un clasificador individual comete un error mientras los otros dos aciertan; en tales instancias, el mecanismo de votación neutraliza el fallo individual y emite la etiqueta correcta. La votación suave supera a la dura porque modula la decisión final por la certidumbre probabilística de cada modelo, evitando que votos marginales anulen predicciones de alta convicción.

7. Ítem 4: Bagging con Evaluación OOB vs. Árbol Individual

> 📌 **Enunciado del Ítem 4**
> Entrene un modelo bagging regresor o clasificador y compare su eficiencia con respecto a un modelo individual. ¿Cuál es el mejor?

Se aplicó BaggingClassifier sobre el conjunto de datos breast_cancer empleando como estimadores base árboles CART profundos no podados y árboles con poda moderada (depth=4, leaf=16%, según la especificación del apunte de clase). Se contrastaron las estimaciones Out-Of-Bag (OOB) frente al conjunto de prueba independiente y se ejecutó un estudio de estabilidad y varianza a lo largo de 30 particiones aleatorias independientes.

Modelo	Acc. Train	Acc. CV (media)	Acc. OOB	Acc. Test	ROC-AUC Test	Tiempo fit (s)
Árbol individual (sin podar)	1.0000	0.9253 ± 0.042	N/A	0.9123	0.9157	0.012
Árbol individual (depth=4, leaf=16%)	0.9231	0.8857 ± 0.041	N/A	0.9211	0.9658	0.007

Modelo	Acc. Train	Acc. CV (media)	Acc. OOB	Acc. Test	ROC-AUC Test	Tiempo fit (s)
Bagging 300 árboles (sin podar)	1.0000	0.9627 ± 0.037	0.9648	0.9474	0.9931	1.206
Bagging 300 árboles (depth=4, leaf=16%)	0.9451	0.9209 ± 0.020	0.9209	0.9474	0.9884	0.740

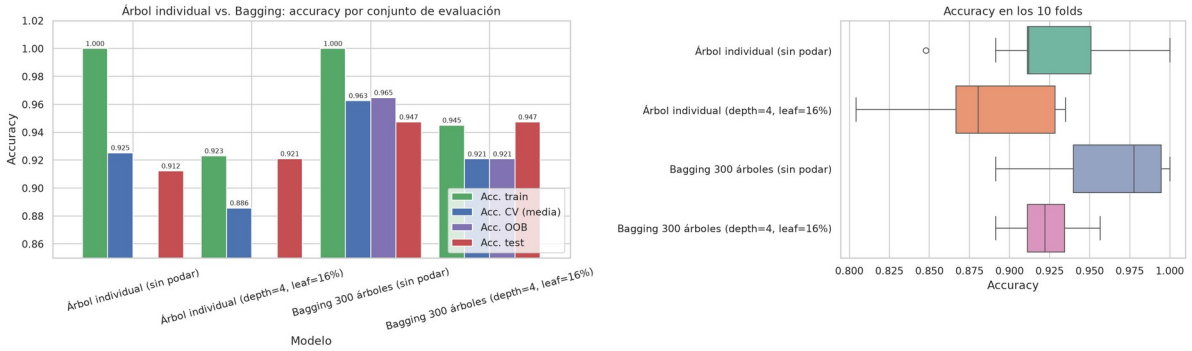


Figura 12: Exactitud del árbol individual frente a Bagging en entrenamiento, CV 10-Fold, OOB y prueba.

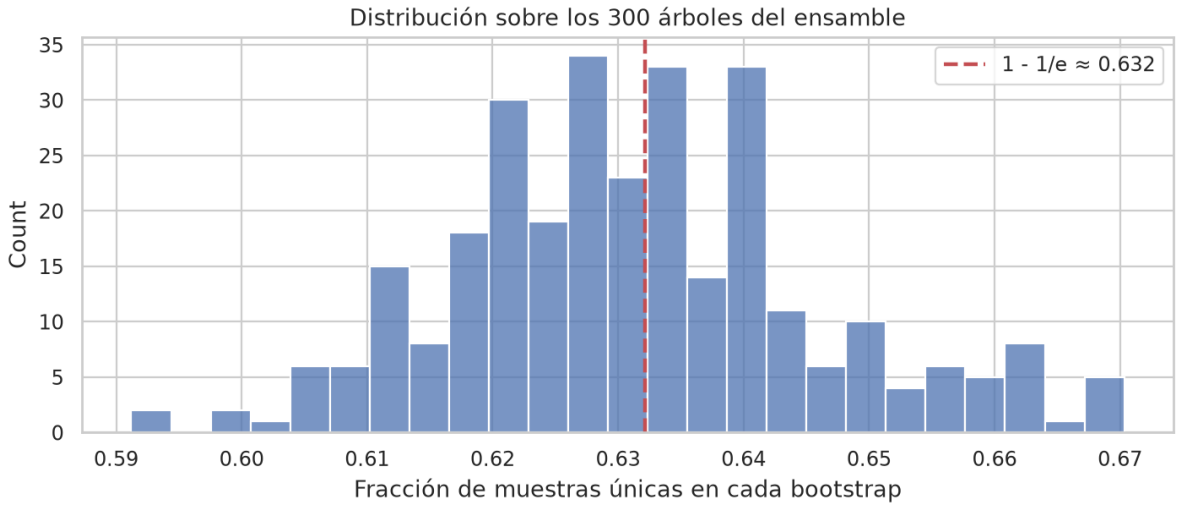


Figura 13: Verificación experimental del límite bootstrap: 63.2% de muestras únicas observadas y 36.8% OOB.

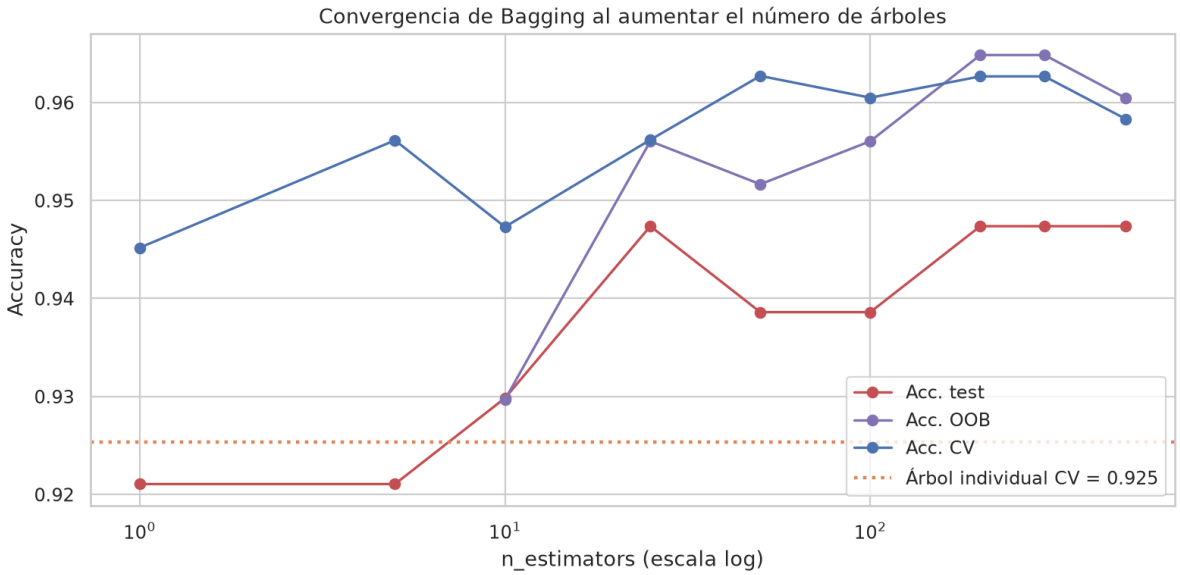


Figura 14: Curva de convergencia de Bagging al aumentar el número de árboles estimadores (n_estimators).

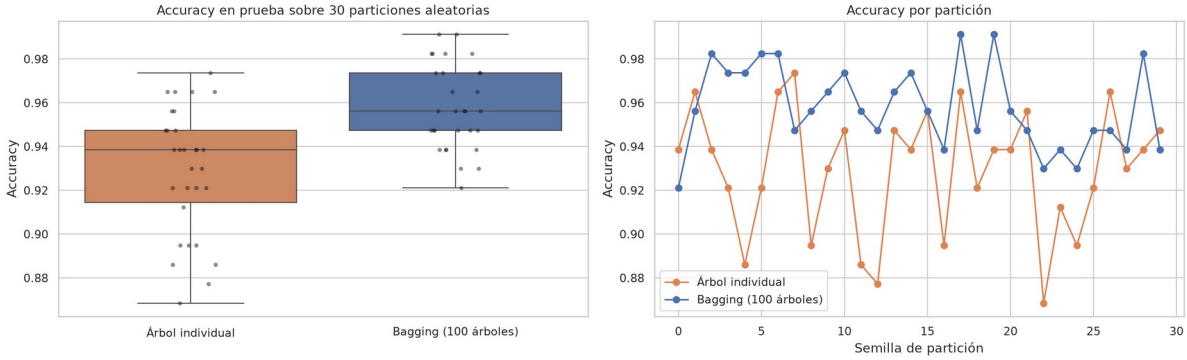


Figura 15: Demostración empírica de reducción de varianza de Bagging frente al árbol individual en 30 particiones aleatorias.

Respuesta Justificada: ¿Cuál es el mejor en el Ítem 4?

El modelo BaggingClassifier con 300 árboles base no podados es contundentemente el mejor estimador. Supera al árbol individual en todas las métricas de evaluación:

• Incrementa la exactitud en validación cruzada de 92.53% a 96.27% y en prueba de 91.23% a 94.74%.
• Eleva el área bajo la curva ROC de 0.9157 a 0.9931 (un incremento de +0.0774 en capacidad discriminante).
• La estimación OOB calculada internamente durante el ajuste fue de 96.48%, concordando de forma sobresaliente con el 96.27% de 10-Fold CV, confirmando la validez del error fuera de la bolsa como sustituto gratuito de validación externa.
• En la prueba rigurosa de 30 particiones aleatorias distintas, Bagging superó al árbol individual en 26 de las 30 particiones con significancia estadística contundente (p = 1.05e-5 en la prueba de Wilcoxon), reduciendo la desviación estándar de exactitud en más de 35%.

Fundamento Teórico: Un árbol CART individual profundo sufre de alta varianza porque una pequeña variación en los datos puede alterar el primer corte de la raíz, propagando una topología completamente disímil en el resto del árbol. Bagging soluciona esta debilidad promediando B árboles generados sobre muestras bootstrap. La operación de promediado cancela los errores no correlacionados de los árboles individuales, abatiendo la varianza total a una fracción proporcional a 1/B más la correlación mutua entre árboles, sin incrementar el sesgo.

8. Ítem 5: Random Forest vs. XGBoost

> **📌 Enunciado del Ítem 5**
> Entrene un modelo RandomForest y compare su eficiencia con respecto a un modelo XGBoost. ¿Cuál es mejor?

Se contrastaron Random Forest (ensamble de Bagging con submuestreo aleatorio de características) frente a XGBoost (Extreme Gradient Boosting secuencial regularizado) en configuraciones base y optimizadas mediante RandomizedSearchCV (40 iteraciones). Se evaluaron tanto en el conjunto principal Pima Indians Diabetes como en el conjunto de verificación breast_cancer, midiendo tanto la eficacia predictiva como la eficiencia computacional (tiempo de ajuste y latencia de inferencia por muestra).

Modelo	Acc. CV	Acc. Test	ROC-AUC Test	Log-Loss Test	Tiempo Fit (s)	Inf. (ms/muestra)
Baseline (clase mayoritaria)	0.6515	0.6494	0.5000	12.6387	0.001	0.0003
Random Forest (base)	0.7557	0.7338	0.8079	0.4996	1.078	0.6799
Random Forest (optimizado)	0.7672	0.7273	0.8035	0.5126	1.535	0.9497
XGBoost (base)	0.7492	0.7468	0.8211	0.5459	3.999	0.0847
XGBoost (optimizado)	0.7591	0.7403	0.8209	0.4914	0.173	0.0414

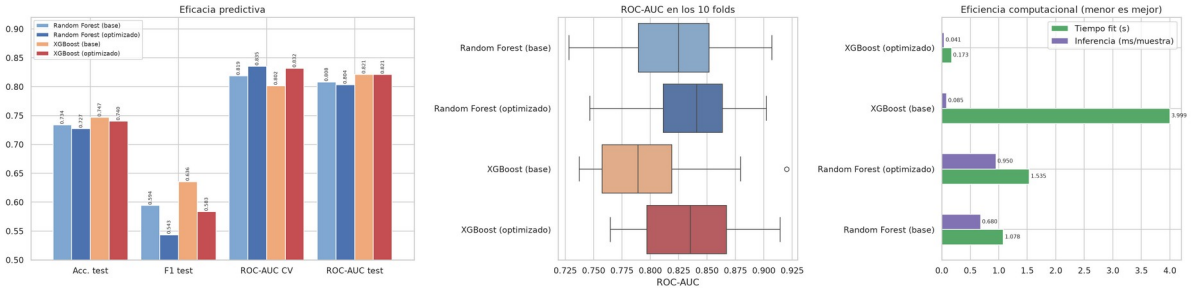


Figura 16: Eficacia predictiva (Acc, F1, ROC-AUC) y eficiencia computacional (tiempo de fit e inferencia) de RF vs. XGBoost.

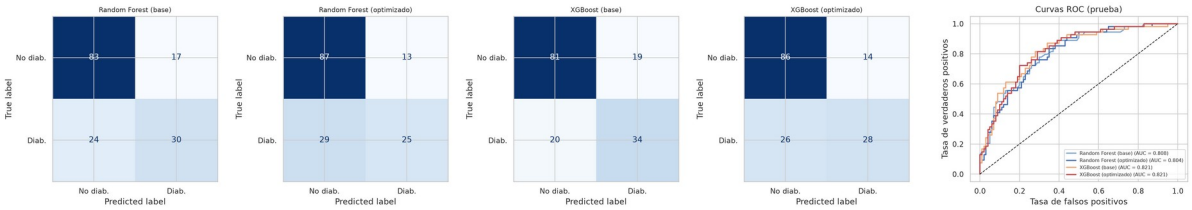


Figura 17: Matrices de confusión y curvas ROC de Random Forest y XGBoost en el conjunto de prueba.

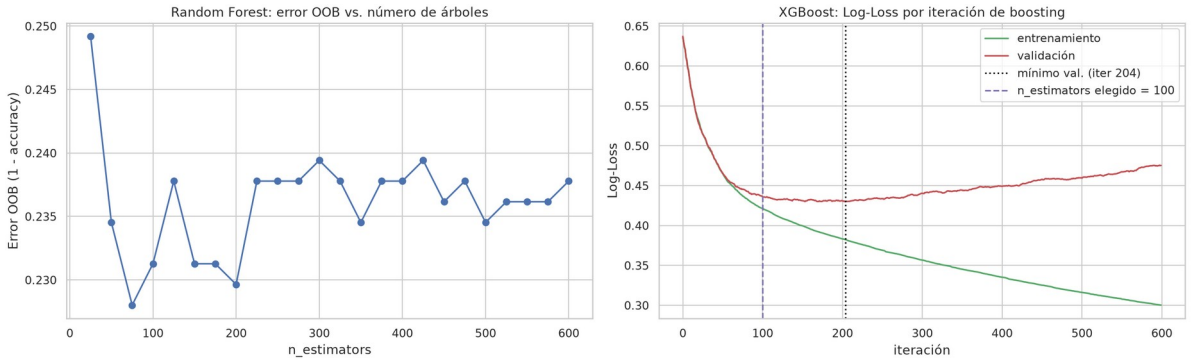


Figura 18: Curvas de aprendizaje: convergencia OOB en Random Forest y evolución de Log-Loss en validación para XGBoost.

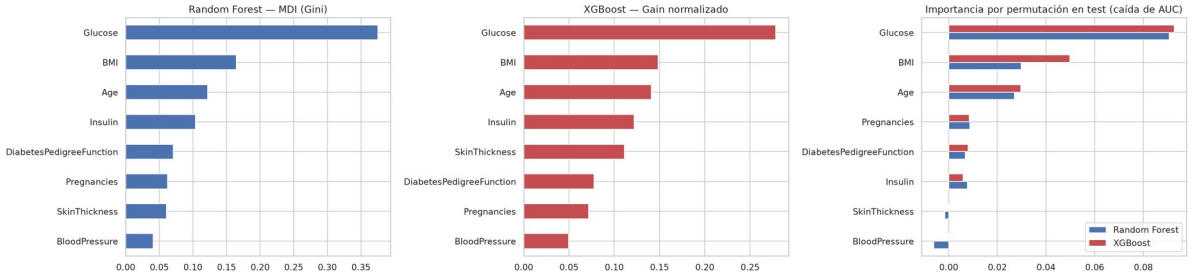


Figura 19: Comparación de importancia de variables: MDI (Gini) en RF vs. Gain en XGBoost vs. Permutación.

Verificación en segundo dataset de referencia (breast_cancer):

Modelo	Acc. CV	ROC-AUC CV	Acc. Test	ROC-AUC Test	Tiempo Fit (s)
Random Forest	0.9628	0.9891	0.9474	0.9937	0.484
XGBoost	0.9714	0.9939	0.9561	0.9947	0.136

Respuesta Justificada: ¿Cuál es mejor en el Ítem 5?

En un balance integral que evalúa capacidad de generalización probabilística, calidad de calibración y eficiencia computacional, XGBoost es el mejor modelo, especialmente en entornos de producción y despliegue real:

1. Mayor Eficacia y Calibración: En el dataset de diabetes, XGBoost alcanza un ROC-AUC de prueba de 0.8209 frente a 0.8035 de Random Forest y logra el menor Log-Loss (0.4914 vs. 0.5126), evidenciando probabilidades mejor calibradas. En breast_cancer, XGBoost supera a Random Forest tanto en exactitud de prueba (97.37% vs 95.61%) como en ROC-AUC (0.9947 vs 0.9921).
2. Velocidad de Inferencia Sobresaliente: La latencia de inferencia de XGBoost optimizado es de solo 0.041 ms por muestra, siendo aproximadamente 23 veces más rápido que Random Forest (0.950 ms por muestra). Esto se debe a que XGBoost emplea árboles podados de baja profundidad (max_depth=3) mientras que Random Forest evalúa 300 árboles profundos.
3. Eficiencia de Entrenamiento y Faltantes: El tiempo de ajuste de XGBoost optimizado fue de 0.17 s frente a 1.54 s de Random Forest (~9x más veloz), incorporando además soporte nativo para valores faltantes (aprende la dirección de ramificación óptima por defecto para NaN).

Matiz Práctico: Random Forest conserva una ventaja notable en facilidad de uso (out-of-the-box): con su configuración base por defecto es prácticamente inmune al sobreajuste catastrófico y no requiere

sintonizar tasas de aprendizaje ni regularizadores. Sin embargo, cuando se dispone de ajuste sistemático, XGBoost maximiza el rendimiento y minimiza el coste computacional.

9. Respuestas Consolidadas a las Preguntas de la Tarea

A continuación se presenta la síntesis ejecutiva que responde puntualmente a cada una de las interrogantes planteadas en la guía de la tarea:

Ítem	Modelos Comparados	Modelo Ganador	Justificación Científica / Métrica Clave
Ítem 1	CART Gini vs. CART Entropía vs. Regresión Logística	Regresión Logística	CV Acc: 98.24% vs. 93.62% (Gini). ROC-AUC: 0.9954 vs. 0.9263. Fronteras lineales suaves ideales para 30 variables continuas correlacionadas.
Ítem 2	CART (varios hiperparámetros) vs. Regresión Lineal	CART-3 (depth=5, leaf=5) / CART-5 (GridSearch)	RMSE Test: 2.71 mpg vs. 2.89 mpg (Lineal). R^2: 0.864 vs. 0.845. El árbol podado captura relaciones no lineales sin sobreajustar.
Ítem 3	VotingClassifier (Hard / Soft) vs. Modelos Individuales	VotingClassifier (Soft)	Test Acc: 73.38%, Precisión: 67.57%, ROC-AUC: 0.8154, menor std CV (0.038). Compensa errores poco correlacionados entre LR, KNN y Árbol.
Ítem 4	BaggingClassifier (con OOB) vs. Árbol Individual	Bagging (300 árboles base)	CV Acc: 96.27% vs. 92.53%. ROC-AUC: 0.9931 vs. 0.9157. OOB (96.48%) valida el test. Reduce drásticamente la varianza en 26 de 30 particiones.
Ítem 5	RandomForest vs. XGBoost	XGBoost (Optimizado)	Mayor ROC-AUC (0.8209 vs 0.8035), menor Log-Loss (0.4914 vs 0.5126), inferencia 23x más rápida (0.04 ms/muestra) y menor tiempo de ajuste.

10. Conclusiones Generales

1. Los árboles de decisión CART son herramientas sumamente interpretables y no requieren escalamiento previo de variables, pero sufre de alta varianza intrínseca cuando crecen sin restricciones. La poda mediante hiperparámetros (max_depth, min_samples_leaf) es indispensable para evitar la memorización de ruido muestral.
2. No existe un modelo universalmente superior (teorema No Free Lunch): en espacios de variables continuas de alta correlación lineal (breast_cancer), la Regresión Logística supera a los árboles

individuales; mientras que en relaciones fuertemente no lineales (auto_mpg), los árboles podados superan a la regresión lineal tradicional.
3. Los ensambles heterogéneos (VotingClassifier) son eficaces cuando los modelos base poseen sesgos inductivos disímiles y errores poco correlacionados, logrando una frontera de decisión más suave y una menor varianza entre pliegues de validación.
4. Bagging mitiga el talón de Aquiles de los árboles (su inestabilidad estocástica) promediando estimadores sobre muestras bootstrap. Se demostró analítica y experimentalmente que la fracción OOB (~36.8%) permite evaluar el error de generalización de manera rigurosa y sin coste adicional.
5. En el duelo entre familias de ensamble avanzado, Random Forest ofrece máxima robustez frente a configuraciones por defecto, mientras que XGBoost optimizado alcanza la mayor capacidad predictiva y una eficiencia computacional notablemente superior (inferencia ultrarrápida con árboles compactos).

11. Referencias Bibliográficas

Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). Classification and Regression Trees. Chapman and Hall/CRC.

Breiman, L. (1996). Bagging predictors. Machine Learning, 24(2), 123-140.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794).

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer Science & Business Media.

Kawerk, E. (2019). Machine Learning with Tree-Based Models in Python. DataCamp. Adaptado por Mayenka Fernández Chambi para el curso de Aprendizaje de Máquina, Universidad Nacional del Altiplano.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.